

Computer Science and Engineering Department  
Thapar Institute of Engineering and Technology, Patiala

# Image Thumbnail Generator

A Cloud-Based Serverless Image Processing System

*Final Prototype Evaluation*

## **Team TechFusion**

Chitvan Goel – [1024030910]  
Sujal Bansal – [1024030659]

[UCS503-SOFTWARE ENGINEERING]  
3C51

## **Faculty Advisor**

Dr.Raghav B. Venkataramaiyer

Computer Science and Engineering Department  
Thapar Institute of Engineering and Technology, Patiala

# Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                  | <b>1</b> |
| 1.1      | Background and Context . . . . .     | 1        |
| 1.2      | Problem Statement . . . . .          | 2        |
| 1.3      | Motivation . . . . .                 | 2        |
| 1.4      | Project Objectives . . . . .         | 3        |
| 1.5      | Scope . . . . .                      | 4        |
|          | 1.5.1 In Scope . . . . .             | 4        |
|          | 1.5.2 Out of Scope . . . . .         | 4        |
| 1.6      | Key Features . . . . .               | 5        |
| <b>2</b> | <b>Methodology and System Design</b> | <b>6</b> |
| 2.1      | System Overview . . . . .            | 6        |
| 2.2      | High-Level Architecture . . . . .    | 7        |
| 2.3      | Component Description . . . . .      | 7        |
|          | 2.3.1 Frontend . . . . .             | 7        |
|          | 2.3.2 PHP Backend . . . . .          | 7        |
|          | 2.3.3 Amazon S3 . . . . .            | 9        |
|          | 2.3.4 AWS Lambda . . . . .           | 9        |
|          | 2.3.5 Pillow . . . . .               | 9        |
|          | 2.3.6 IAM . . . . .                  | 9        |
| 2.4      | Technical Stack . . . . .            | 10       |
| 2.5      | End-to-End Workflow . . . . .        | 10       |
| 2.6      | Image Processing Logic . . . . .     | 12       |
| 2.7      | Security Design . . . . .            | 13       |
| 2.8      | Event-Driven Architecture . . . . .  | 15       |

|          |                                        |           |
|----------|----------------------------------------|-----------|
| <b>3</b> | <b>Implementation</b>                  | <b>17</b> |
| 3.1      | Frontend Implementation . . . . .      | 17        |
| 3.2      | PHP Upload Implementation . . . . .    | 17        |
| 3.3      | Lambda Implementation . . . . .        | 18        |
| 3.4      | Presigned URL Implementation . . . . . | 19        |
| 3.5      | Thumbnail Status Checking . . . . .    | 20        |
| 3.6      | Error Handling . . . . .               | 20        |
| <b>4</b> | <b>Results and Evaluation</b>          | <b>21</b> |
| 4.1      | Prototype Demonstration . . . . .      | 21        |
| 4.2      | Functional Testing . . . . .           | 24        |
| 4.3      | Integration Testing . . . . .          | 24        |
| 4.4      | Performance Discussion . . . . .       | 25        |
| 4.5      | Results Analysis . . . . .             | 26        |
| <b>5</b> | <b>Limitations and Future Scope</b>    | <b>28</b> |
| 5.1      | Current Limitations . . . . .          | 28        |
| 5.2      | Future Enhancements . . . . .          | 29        |
| <b>6</b> | <b>Conclusion</b>                      | <b>30</b> |

# List of Figures

|     |                                                                                                                                                                              |    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | High-level architecture of the Image Thumbnail Generator, showing the flow from browser upload through S3-triggered Lambda processing to secure thumbnail retrieval. . . . . | 8  |
| 2.2 | End-to-end workflow: from browser upload to secure thumbnail retrieval. . . . .                                                                                              | 12 |
| 4.1 | Final upload interface of the Image Thumbnail Generator. . .                                                                                                                 | 21 |
| 4.2 | Client-side preview of a selected image prior to upload. . . .                                                                                                               | 22 |
| 4.3 | Successful upload confirmation displayed to the user. . . . .                                                                                                                | 22 |
| 4.4 | Thumbnail generated by the Lambda function and displayed in the browser. . . . .                                                                                             | 23 |
| 4.5 | Original image compared against its generated thumbnail. . .                                                                                                                 | 23 |

# List of Tables

|     |                                                             |    |
|-----|-------------------------------------------------------------|----|
| 2.1 | Technical stack used in the Image Thumbnail Generator . . . | 10 |
| 4.1 | Functional test cases and observed results . . . . .        | 24 |
| 4.2 | Performance factors relevant to future evaluation . . . . . | 26 |

# Chapter 1

## Introduction

### 1.1 Background and Context

The past decade has seen an enormous rise in the number of web applications that are fundamentally centered around images, including e-commerce catalogs, social media platforms, content management systems, and portfolio websites. As the volume of user-generated and application-hosted image content has grown, so too has the importance of delivering that content efficiently to end users. Displaying full-resolution images in contexts such as listing pages, search results, or grid galleries is both unnecessary and wasteful: a full-sized photograph captured by a modern camera or smartphone may occupy several megabytes, while the space it occupies on screen in a thumbnail view is often only a few square centimeters.

This mismatch between the size of stored images and the size at which they are actually displayed has direct consequences for storage costs, network bandwidth consumption, and, most importantly, page load performance and user experience. Consequently, the generation of appropriately sized *thumbnails* — smaller, optimized derivative images — has become a standard requirement in modern web application design.

Historically, thumbnail generation has been handled either manually, by requiring content owners to upload pre-resized images, or synchronously, by processing images directly within the same server process that handles the user's upload request. Both approaches present limitations. Manual thumbnail creation is tedious, error-prone, and does not scale as the number of uploaded images increases. Synchronous, in-process resizing places addi-

tional CPU and memory load directly on the web application server at the exact moment a user is interacting with it, which can degrade responsiveness, particularly under concurrent upload load.

These limitations motivate the exploration of an alternative architecture in which image processing is decoupled from the main application server and instead delegated to a cloud-based, event-driven processing pipeline. This report documents the design, implementation, and evaluation of exactly such a system: the **Image Thumbnail Generator**, developed by team TechFusion.

## 1.2 Problem Statement

The central problem addressed by this project can be stated as follows:

*How can image thumbnail generation be automated through a cloud-based architecture without placing the image-processing workload directly on the main web application server?*

A naive solution might resize images synchronously within the same PHP request that handles the file upload. However, this approach ties the responsiveness of the web application directly to the computational cost of image processing, and does not scale gracefully as image sizes or upload volumes increase. The project therefore investigates an event-driven, serverless alternative in which the act of storing an image and the act of processing that image are separated into independent, loosely coupled stages, connected through a cloud storage event notification mechanism rather than a direct function call.

## 1.3 Motivation

Several considerations motivated the specific architectural choices made in this project:

- **Automation.** Removing the need for any manual thumbnail creation step, so that a thumbnail is always produced automatically whenever a new image is uploaded.
- **Reduced application-server processing.** Ensuring that the PHP backend is responsible only for validation, storage orchestration, and

metadata/URL handling, not for the computationally heavier task of image manipulation.

- **Cloud storage as the system of record.** Using Amazon S3 as a durable, centrally accessible store for both original images and generated thumbnails, rather than relying on local server disk storage.
- **Event-driven processing.** Leveraging S3's native event notification capability to trigger processing automatically, without polling or manual invocation.
- **Scalability potential.** Because AWS Lambda scales automatically with the number of incoming events, the architecture has the potential to handle bursts of uploads without requiring the web server itself to be scaled.
- **Secure image access.** Ensuring that images stored in a private S3 bucket are only made accessible to end users through time-limited, presigned URLs rather than permanent public links.

## 1.4 Project Objectives

The following objectives were defined for the final prototype:

1. Develop a web interface that allows a user to upload an image.
2. Validate uploaded images securely, both on the client side and on the server side.
3. Store original uploaded images in Amazon S3.
4. Automatically trigger image processing through S3 event notifications.
5. Process uploaded images using an AWS Lambda function written in Python, using the Pillow library.
6. Generate a thumbnail while preserving the original image's aspect ratio.
7. Store the generated thumbnail separately from the original image within Amazon S3.



8. Provide secure, time-limited access to both original images and thumbnails using presigned S3 URLs.
9. Integrate all of the above into a single, functional, end-to-end prototype.

## 1.5 Scope

### 1.5.1 In Scope

The current prototype includes:

- A browser-based upload interface supporting drag-and-drop and traditional file selection.
- Client-side and server-side validation of uploaded files.
- Upload of validated original images to a designated `uploads/` prefix in an Amazon S3 bucket.
- An S3 event notification configured on the `uploads/` prefix that triggers an AWS Lambda function.
- A Python-based Lambda function that uses Pillow to generate a single resized thumbnail while preserving aspect ratio.
- Storage of the generated thumbnail under a separate `thumbnails/` prefix in the same S3 bucket.
- A status-checking mechanism by which the frontend/backend determines when the thumbnail has become available.
- Presigned URL generation for secure, temporary access to S3 objects.

### 1.5.2 Out of Scope

The current prototype does *not* include, and these are explicitly identified as future scope rather than existing functionality:

- User authentication or account management.

- Generation of multiple thumbnail sizes per image.
- Batch or bulk image processing.
- A production-grade monitoring or logging dashboard.
- Content delivery network (CDN) integration.
- Automated S3 lifecycle management policies.

## 1.6 Key Features

- **Drag-and-drop upload** supported directly in the browser interface.
- **Client-side file preview**, allowing the user to see the selected image before it is uploaded.
- **Client-side and server-side validation** of file type and size.
- **Direct integration with Amazon S3** for durable, centralized storage of both original images and thumbnails.
- **Event-driven Lambda processing**, triggered automatically upon successful upload, without any explicit invocation from the PHP backend.
- **Automatic thumbnail generation** with aspect-ratio preservation, implemented using the Pillow imaging library.
- **Status polling** from the frontend/backend to determine when an asynchronously generated thumbnail becomes available.
- **Secure, presigned URL access** to both original and thumbnail images, avoiding the need for a publicly readable S3 bucket.
- **Structured error handling** across the validation, upload, and processing stages of the pipeline.

# Chapter 2

## Methodology and System Design

### 2.1 System Overview

The Image Thumbnail Generator is structured as a small number of loosely coupled components that communicate primarily through Amazon S3, rather than through direct synchronous calls between the web application and the image-processing logic. This design reflects an intentional architectural decision: by using S3 as the point of hand-off between the PHP backend and the Lambda-based processing stage, the two halves of the system can be developed, deployed, and scaled independently of one another.

At a high level, the system can be divided into three logical layers:

1. A **presentation layer**, consisting of the HTML/CSS/JavaScript frontend served to the user's browser.
2. An **application layer**, consisting of the PHP 8 backend responsible for validation, S3 interaction, and presigned URL generation.
3. A **processing layer**, consisting of the AWS Lambda function and the Pillow-based image transformation logic that it executes.

## 2.2 High-Level Architecture

Figure 2.1 illustrates the overall flow of data through the system, from the moment a user selects an image in their browser to the moment a generated thumbnail is displayed back to them.

The critical architectural property illustrated in Figure 2.1 is that the PHP application never directly invokes the image-processing logic. Instead, the act of placing an object into the `uploads/` prefix of the S3 bucket is itself sufficient to trigger processing, via S3’s native event notification system. This decoupling is central to the event-driven design philosophy of the project and is discussed further in Section 2.8.

## 2.3 Component Description

### 2.3.1 Frontend

The frontend is implemented using plain HTML, CSS, and vanilla JavaScript, without any external frontend framework. It is responsible for presenting the upload interface, supporting both drag-and-drop and traditional file-picker interactions, generating an immediate client-side preview of the selected image, and performing lightweight validation (such as checking the file’s MIME type and size) before the file is transmitted to the server. Communication with the PHP backend is performed asynchronously using the Fetch API, allowing the interface to remain responsive while the upload is in progress. Once the original image has been uploaded, the frontend transitions into a polling state, periodically checking whether the corresponding thumbnail has become available.

### 2.3.2 PHP Backend

The backend is implemented in PHP 8 [7] and makes use of the official AWS SDK for PHP [4] to interact with Amazon S3. Its responsibilities include receiving the uploaded file, performing authoritative server-side validation (including MIME-type detection using PHP’s `finfo` extension), generating a randomized, collision-resistant filename, uploading the validated original image to the `uploads/` prefix of the S3 bucket, and later generating presigned URLs so that the browser can retrieve both the original image and, once available, its thumbnail.

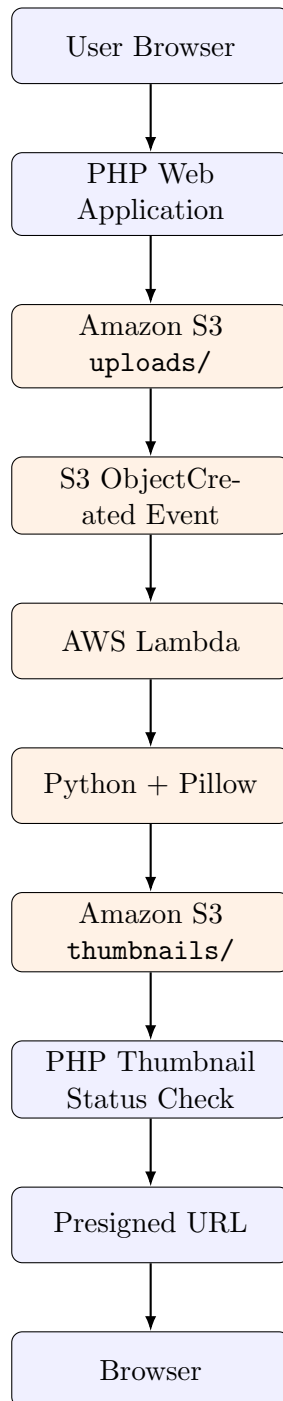


Figure 2.1: High-level architecture of the Image Thumbnail Generator, showing the flow from browser upload through S3-triggered Lambda processing to secure thumbnail retrieval.

### 2.3.3 Amazon S3

Amazon S3 serves as the central, durable object store for the system [1]. Two logical prefixes are used within a single bucket: `uploads/`, which holds original images as submitted by users, and `thumbnails/`, which holds the corresponding Lambda-generated thumbnails. The bucket is configured as private, with access to individual objects granted only through presigned URLs rather than through public bucket policies. An S3 event notification is configured on the `uploads/` prefix so that any new object created there automatically invokes the processing Lambda function.

### 2.3.4 AWS Lambda

AWS Lambda hosts the image-processing logic in a fully serverless manner [2]: no server is provisioned or managed by the project team, and the function executes only in response to an incoming S3 event. The Lambda function is implemented in Python and receives, as part of its invocation event, the bucket name and object key of the newly uploaded image. It uses this information to retrieve the image from S3, process it, and write the resulting thumbnail back to S3.

### 2.3.5 Pillow

Pillow [6], the Python Imaging Library fork, is used within the Lambda function to perform the actual image manipulation. Its responsibilities include opening the downloaded image file, normalizing its color mode where necessary (for example, converting palette-based or RGBA images into a mode suitable for JPEG output), computing new dimensions that preserve the original aspect ratio, performing the resize operation, and saving the resulting thumbnail in the target format.

### 2.3.6 IAM

AWS Identity and Access Management (IAM) [3] is used to define the permissions under which the Lambda function and the PHP backend operate. Following the principle of least privilege, the Lambda function's execution role is scoped to only the S3 read and write permissions it requires on the

relevant bucket prefixes, and the credentials used by the PHP backend are similarly scoped rather than granted broad administrative access.

## 2.4 Technical Stack

Table 2.1 summarizes the technologies used at each layer of the system.

Table 2.1: Technical stack used in the Image Thumbnail Generator

| Layer                 | Technology                  | Purpose                                                               |
|-----------------------|-----------------------------|-----------------------------------------------------------------------|
| Frontend              | HTML / CSS / JavaScript     | User interface, upload interaction, preview                           |
| Backend               | PHP 8                       | Upload handling and AWS integration                                   |
| Storage               | Amazon S3                   | Storage of original images and thumbnails                             |
| Processing            | AWS Lambda                  | Serverless, event-driven image processing                             |
| Image Library Runtime | Pillow (PIL)<br>Python 3.12 | Image transformation and resizing<br>Execution environment for Lambda |
| Security              | AWS IAM                     | Access control and least-privilege permissions                        |
| Access                | Presigned URLs              | Secure, time-limited object retrieval                                 |

## 2.5 End-to-End Workflow

The complete request-to-response workflow proceeds as follows:

1. The user selects an image, either by dragging it onto the drop zone or through the file picker.
2. Client-side JavaScript performs an initial validation of file type and size.

3. A preview of the selected image is displayed to the user.
4. The file is transmitted to the PHP backend using an asynchronous request.
5. The PHP backend performs authoritative server-side validation, including MIME-type detection.
6. A unique, randomized filename is generated for the object.
7. The original image is uploaded to the `uploads/` prefix of the S3 bucket.
8. The successful `PutObject` operation causes S3 to emit an `ObjectCreated` event.
9. This event invokes the AWS Lambda function.
10. The Lambda function downloads the newly uploaded image from S3.
11. Pillow opens the downloaded image within the Lambda execution environment.
12. The image's original dimensions are read and evaluated.
13. Target thumbnail dimensions are calculated so as to preserve the original aspect ratio.
14. The image is resized accordingly.
15. The resized image is converted and saved in the appropriate output format.
16. The resulting thumbnail is uploaded to the `thumbnails/` prefix of the same S3 bucket.
17. The frontend, via the PHP backend, polls to check whether the thumbnail object now exists.
18. Once available, the PHP backend generates a presigned URL for the thumbnail.
19. The browser uses this presigned URL to display the generated thumbnail to the user.



Figure ?? presents this same sequence in diagrammatic form, emphasizing the asynchronous boundary between the upload path and the processing path.

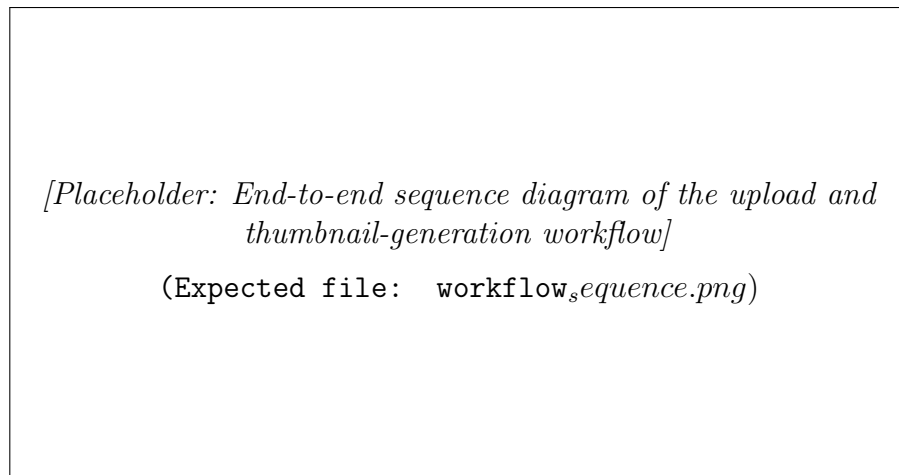


Figure 2.2: End-to-end workflow: from browser upload to secure thumbnail retrieval.

## 2.6 Image Processing Logic

The core responsibility of the Lambda function is to transform a source image into a smaller thumbnail without distorting its proportions. The logic can be summarized as follows:

1. Read the source image's width and height.
2. Determine a maximum target dimension (for example, a maximum width or height in pixels).
3. Compute a scaling factor such that the larger of the two source dimensions is reduced to the target dimension, and apply the same scaling factor to the other dimension, thereby preserving the original aspect ratio.
4. Avoid unnecessarily enlarging images that are already smaller than the target thumbnail size.

5. Normalize the image mode where required — for example, converting an image in palette mode (as is common with GIF input) or RGBA mode into RGB, so that it can be saved as a JPEG.
6. Perform the resize operation using Pillow’s resampling functionality.
7. Save the resulting thumbnail, and upload it to the designated `thumbnails/` prefix in S3.

Listing 2.1 presents simplified pseudocode summarizing this logic. The actual implementation includes additional error handling, described in Section 3.6.

```
1 def generate_thumbnail(image_path, max_dimension):
2     image = Image.open(image_path)
3
4     # Normalize color mode for consistent JPEG output
5     if image.mode in ("P", "RGBA"):
6         image = image.convert("RGB")
7
8     width, height = image.size
9     scale = min(max_dimension / width, max_dimension / height
10                , 1.0)
11     new_size = (int(width * scale), int(height * scale))
12
13     image.thumbnail(new_size)
14     image.save(output_path, format="JPEG")
15     return output_path
```

Listing 2.1: Simplified thumbnail-generation logic (pseudocode)

This report does not claim support for image formats or processing behaviors beyond what has actually been implemented and verified in the prototype; formats not explicitly tested are noted as such in Section 4.2.

## 2.7 Security Design

Security was treated as a first-class design concern throughout the project, rather than an afterthought. The following measures were incorporated into the prototype:

- **Environment variables for AWS credentials.** AWS access keys and related configuration values are supplied to the application through

environment variables rather than being hard-coded into source files, reducing the risk of accidental credential exposure.

- **Exclusion of `.env` from version control.** The environment configuration file is explicitly excluded from the project's version control repository so that credentials are never committed to source history.
- **Private S3 bucket.** The S3 bucket used for both original images and thumbnails is kept private; no objects are made publicly readable.
- **Presigned URLs.** Access to individual objects is granted only through time-limited presigned URLs generated by the PHP backend, ensuring that access is both temporary and scoped to a specific object.
- **Client-side validation.** Basic file-type and size checks are performed in the browser to provide immediate feedback and reduce unnecessary upload attempts.
- **Server-side validation.** Because client-side checks can be bypassed, the PHP backend independently re-validates every uploaded file.
- **MIME detection using `finfo`.** Rather than trusting the file extension or client-supplied content type, the backend inspects the actual file contents using PHP's `finfo` extension to determine the true MIME type.
- **File extension validation.** Uploaded files are additionally checked against an allow-list of accepted extensions.
- **Maximum file size validation.** Uploads exceeding a configured size limit are rejected before being written to storage.
- **Randomized filenames.** Generated filenames do not reuse client-supplied names, reducing the risk of filename collisions and certain classes of injection or overwrite attacks.
- **Path traversal protection.** User-supplied filenames are never used directly to construct storage paths, preventing directory traversal attempts.

- **IAM least-privilege permissions.** Both the Lambda execution role and the credentials used by the PHP backend are scoped to the minimum S3 permissions required for their respective roles.
- **Avoidance of unnecessary public access.** No S3 bucket policy or object ACL grants public access; all retrieval is mediated through presigned URLs.

## 2.8 Event-Driven Architecture

The decision to use an event-driven, serverless architecture for the processing stage of the system was central to the design of this project. Rather than the PHP application directly invoking a resizing routine, the system relies on Amazon S3’s built-in event notification mechanism: when a new object is created under the `uploads/` prefix, S3 automatically emits an event that invokes the Lambda function.

This design offers several advantages:

- **Decoupling.** The upload path and the processing path are independent; the PHP application has no direct dependency on the Lambda function’s implementation.
- **Asynchronous processing.** The user’s upload request can complete as soon as the original image has been stored, without waiting for thumbnail generation to finish.
- **Serverless execution.** No dedicated server needs to be provisioned, patched, or scaled to run the image-processing logic.
- **Reduced workload on the PHP server.** CPU- and memory-intensive image manipulation is offloaded entirely to the Lambda execution environment.
- **Scalability potential.** Because Lambda invocations scale automatically with the number of incoming S3 events, the processing layer has the potential to absorb bursts of uploads without manual intervention.

An important detail of this design is the use of *separate* S3 prefixes for original images and generated thumbnails. If thumbnails were written

back into the same `uploads/` prefix that triggers the Lambda function, each newly created thumbnail would itself satisfy the trigger condition, causing the Lambda function to be invoked recursively and attempt to generate a “thumbnail of a thumbnail” indefinitely. By configuring the S3 event notification so that it fires only for objects created under `uploads/`, and having the Lambda function write its output to the distinct `thumbnails/` prefix, this recursive-trigger scenario is avoided by construction.

# Chapter 3

## Implementation

### 3.1 Frontend Implementation

The upload interface was implemented using plain HTML and CSS for structure and styling, together with vanilla JavaScript for behavior, without introducing an external frontend framework. The interface exposes both a drag-and-drop drop zone and a conventional file-input control, so that users may select an image using whichever interaction they find more convenient.

Upon selection, JavaScript reads the file using the browser's File API and immediately renders a preview using an object URL, allowing the user to confirm the correct file was chosen before it is transmitted to the server. Basic client-side checks (file type and size) are performed at this stage to provide fast feedback, understanding that these checks are advisory only and are re-verified authoritatively on the server.

Once the user confirms the upload, the file is transmitted to the PHP backend using the Fetch API as part of a multipart form submission. The interface then displays a loading/status indicator and begins polling a status endpoint to determine when the corresponding thumbnail has been generated, at which point the interface updates to display the thumbnail image retrieved via its presigned URL.

### 3.2 PHP Upload Implementation

The PHP backend receives the uploaded file through the standard `$_FILES` superglobal, performs validation, and uses the AWS SDK for PHP to upload

the validated file to Amazon S3. Listing 3.1 illustrates the general structure of this logic in simplified form.

```
1 $file = $_FILES['image'];
2
3 $finfo = new finfo(FILEINFO_MIME_TYPE);
4 $mimeType = $finfo->file($file['tmp_name']);
5
6 if (!in_array($mimeType, $allowedMimeTypes, true)) {
7     respondWithError('Unsupported file type.');
```

```
8 }
9
10 if ($file['size'] > $maxUploadSizeBytes) {
11     respondWithError('File exceeds maximum allowed size.');
```

```
12 }
13
14 $safeFilename = bin2hex(random_bytes(16)) . '.jpg';
15 $s3Key = 'uploads/' . $safeFilename;
16
17 $s3Client->putObject([
18     'Bucket' => $bucketName,
19     'Key' => $s3Key,
20     'SourceFile' => $file['tmp_name'],
21 ]);
```

Listing 3.1: Simplified PHP upload handling logic

Key aspects of this implementation include independent server-side MIME-type detection (rather than trusting the client-supplied content type), enforcement of a maximum file size, and generation of a randomized filename that is never derived from user-supplied input, thereby avoiding path traversal and filename-collision issues.

### 3.3 Lambda Implementation

The AWS Lambda function is implemented in Python [5] and is invoked automatically by the S3 event notification configured on the `uploads/` prefix. Its handler extracts the relevant bucket name and object key from the incoming event, downloads the corresponding object to a temporary location within the Lambda execution environment, processes it using Pillow, and uploads the resulting thumbnail back to S3. Listing 3.2 presents a simplified, representative version of this handler.

```

1 import boto3
2 from PIL import Image
3
4 s3 = boto3.client('s3')
5
6 def lambda_handler(event, context):
7     record = event['Records'][0]['s3']
8     bucket = record['bucket']['name']
9     key = record['object']['key']
10
11     download_path = '/tmp/source_image'
12     upload_key = key.replace('uploads/', 'thumbnails/')
13
14     try:
15         s3.download_file(bucket, key, download_path)
16         thumbnail_path = generate_thumbnail(download_path,
17                                             MAX_DIMENSION)
18         s3.upload_file(thumbnail_path, bucket, upload_key)
19     except Exception as error:
20         # Processing failure is logged; see Section 3.6
21         raise error

```

Listing 3.2: Simplified Lambda handler logic

Temporary files created during processing reside in the Lambda execution environment’s ephemeral `/tmp` storage, which is discarded automatically at the end of each invocation, so no explicit cleanup of the container filesystem is required by the function itself.

## 3.4 Presigned URL Implementation

Because the S3 bucket used by the system is private, objects cannot be retrieved through direct, permanent URLs. Instead, the PHP backend uses the AWS SDK for PHP to generate a presigned URL: a specially signed, time-limited URL that grants temporary read access to a specific object without requiring the requester to hold AWS credentials of their own. The backend generates such a URL on demand, both for the original uploaded image (where needed) and for the generated thumbnail once it becomes available, and returns this URL to the frontend for direct use in an `<img>` element.



## 3.5 Thumbnail Status Checking

Because thumbnail generation occurs asynchronously — potentially several seconds after the original image has finished uploading — the frontend cannot assume that the thumbnail is immediately available. Instead, after the original upload completes, the frontend repeatedly issues lightweight status-check requests to the PHP backend at a fixed interval. Each such request causes the backend to check, via the AWS SDK, whether an object now exists at the corresponding key under the `thumbnails/` prefix. Once the object is found, the backend responds with a presigned URL for the thumbnail, and the frontend stops polling and renders the image.

## 3.6 Error Handling

The prototype anticipates and handles the following categories of error:

- **Invalid file type.** Rejected at both the client (for immediate feedback) and the server (authoritatively, using MIME detection).
- **Oversized file.** Rejected by the server if the uploaded file exceeds the configured maximum size.
- **Failed upload to S3.** If the `putObject` call to S3 fails, the backend returns an error response to the frontend rather than reporting a false success.
- **S3 or Lambda service failure.** Failures within the Lambda function are caught and logged (via the function’s own exception handling and the AWS Lambda execution environment’s logging), rather than allowed to fail silently.
- **Invalid or corrupt image data.** If Pillow is unable to open the downloaded file as a valid image, the Lambda function raises an exception rather than producing a malformed thumbnail.
- **Thumbnail not yet available.** While polling, if the thumbnail object does not yet exist, the backend responds indicating that processing is still in progress, rather than treating this as an error condition.

# Chapter 4

## Results and Evaluation

### 4.1 Prototype Demonstration

The final prototype was exercised through its complete workflow: a user selects an image via the browser interface, the image is uploaded and stored in Amazon S3, the Lambda function is triggered automatically, a thumbnail is generated and stored separately, and the frontend retrieves and displays the resulting thumbnail through a presigned URL. Figures 4.1 through 4.5 illustrate this workflow as demonstrated in the final prototype.

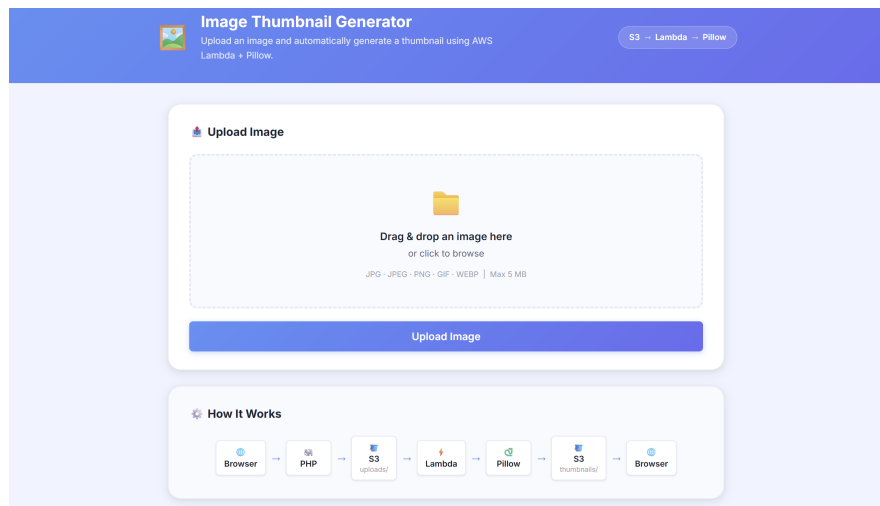


Figure 4.1: Final upload interface of the Image Thumbnail Generator.

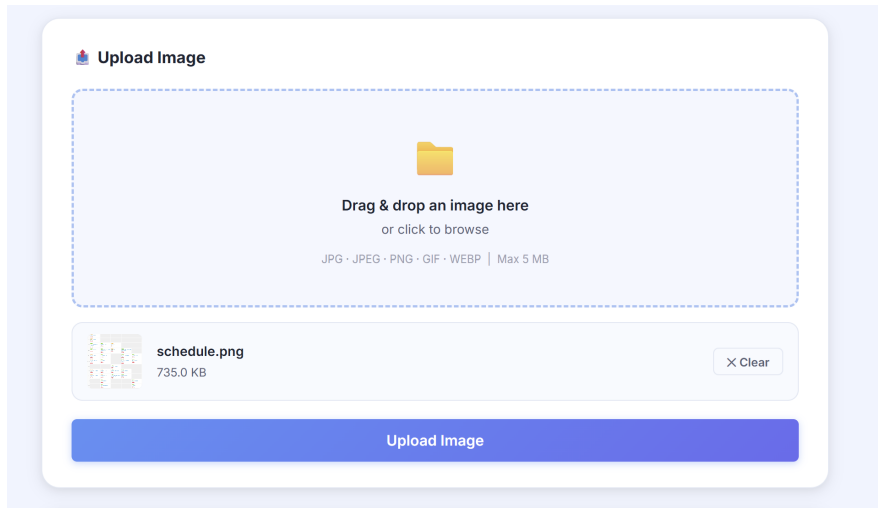


Figure 4.2: Client-side preview of a selected image prior to upload.

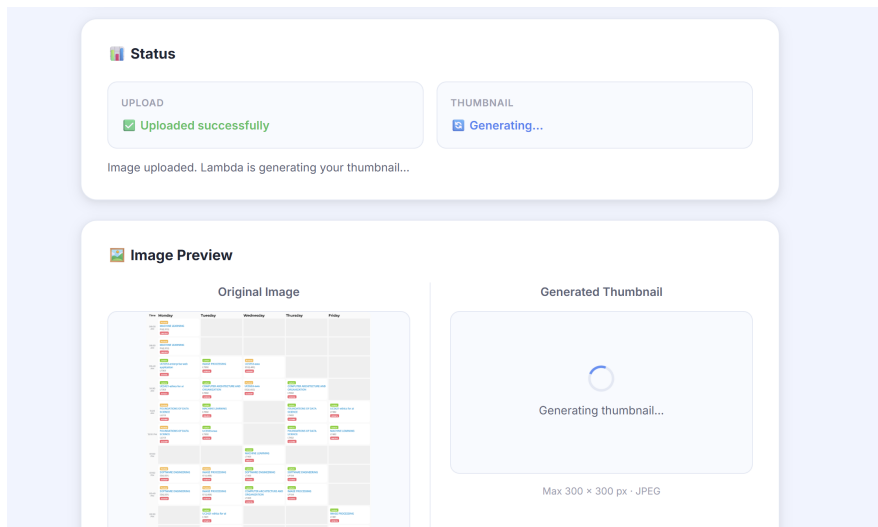


Figure 4.3: Successful upload confirmation displayed to the user.

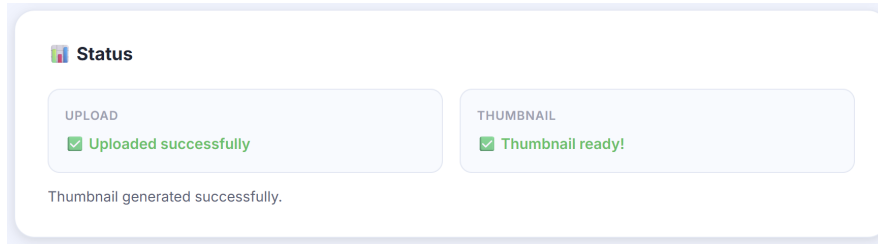


Figure 4.4: Thumbnail generated by the Lambda function and displayed in the browser.

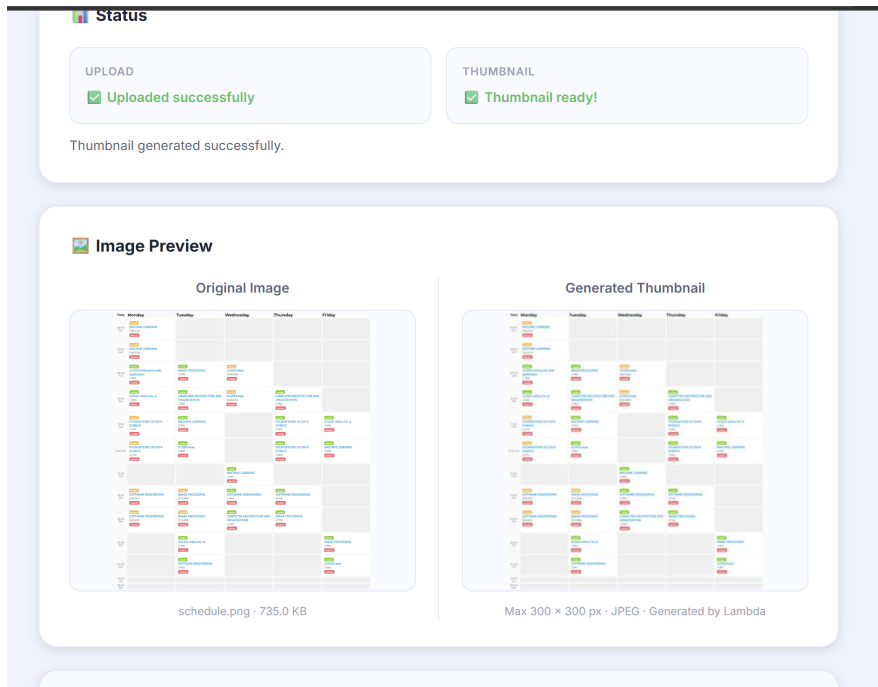


Figure 4.5: Original image compared against its generated thumbnail.

## 4.2 Functional Testing

Table 4.1 summarizes the functional test cases exercised against the prototype. Only outcomes that were actually observed are reported as “Pass”; any behavior that has not yet been explicitly verified is marked accordingly rather than assumed.

Table 4.1: Functional test cases and observed results

| Test Case            | Input / Condition                    | Expected Result               | Status                      |
|----------------------|--------------------------------------|-------------------------------|-----------------------------|
| Valid JPG upload     | Valid JPEG image within size limit   | Upload succeeds               | Pass                        |
| Valid PNG upload     | Valid PNG image                      | Thumbnail generated           | Pass                        |
| Invalid file type    | Unsupported file extension/MIME type | Upload rejected               | Pass                        |
| Oversized file       | File exceeding configured size limit | Upload rejected               | Pass                        |
| Thumbnail generation | Valid successful upload              | Lambda creates thumbnail      | Pass                        |
| Secure access        | Private S3 object requested          | Presigned URL generated       | Pass                        |
| GIF input handling   | GIF image uploaded                   | Thumbnail generated correctly | Not Tested / To Be Verified |
| Concurrent uploads   | Multiple simultaneous uploads        | All processed independently   | Not Tested / To Be Verified |

## 4.3 Integration Testing

In addition to the functional test cases above, the following integration points within the pipeline were exercised:

- **Frontend** → **PHP**. Confirmed that files selected in the browser are correctly received by the PHP upload endpoint.
- **PHP** → **S3**. Confirmed that validated files are correctly written to the expected `uploads/` key in the S3 bucket.
- **S3** → **Lambda**. Confirmed that the configured event notification correctly invokes the Lambda function upon object creation.
- **Lambda** → **S3**. Confirmed that the Lambda function correctly writes the generated thumbnail to the `thumbnails/` prefix.
- **PHP** → **generated thumbnail**. Confirmed that the PHP status-check endpoint correctly detects the presence of a newly created thumbnail object.
- **Browser** → **presigned URL**. Confirmed that presigned URLs returned by the backend can be used directly by the browser to retrieve the corresponding image.

## 4.4 Performance Discussion

No formal load testing or benchmarking has been performed on the current prototype, and this report accordingly does not present fabricated latency or throughput figures. Instead, several qualitative factors relevant to the system’s performance characteristics are discussed below.

Because thumbnail generation is performed asynchronously via Lambda rather than synchronously within the PHP request, the perceived responsiveness of the upload interface is decoupled from the actual duration of image processing: the user’s upload request can complete as soon as the original file has been written to S3. The overall latency experienced by the user before a thumbnail becomes visible is influenced by several factors, including the size of the uploaded image, the propagation delay of the S3 event notification, Lambda cold-start behavior (if the function has not been invoked recently), and the polling interval used by the frontend.

The separation of original images and thumbnails into distinct S3 prefixes, combined with the serverless nature of the Lambda processing tier, also provides a reasonable basis for future scalability: because Lambda automatically provisions additional concurrent execution environments in response

to incoming events, the processing tier has the potential to accommodate increased upload volume without requiring the PHP application server itself to be scaled. Table 4.2 indicates the categories of performance data that would be relevant to formally characterize in future evaluation work, with placeholders where the current prototype does not yet report measured values.

Table 4.2: Performance factors relevant to future evaluation

| <b>Metric</b>                                               | <b>Current Status</b>       |
|-------------------------------------------------------------|-----------------------------|
| End-to-end processing latency (upload to visible thumbnail) | [Insert measured value]     |
| Lambda cold-start duration                                  | [Insert measured value]     |
| Average uploaded image size                                 | [Insert measured value]     |
| Concurrent upload handling capacity                         | Not Tested / To Be Verified |

## 4.5 Results Analysis

Against the objectives defined in Section 1.4, the final prototype successfully demonstrates the complete intended workflow: a user can upload an image through the web interface, have that image validated and stored in Amazon S3, trigger automatic processing via an S3 event, generate a thumbnail using AWS Lambda and Pillow, and securely retrieve that thumbnail via a presigned URL.

Several implementation challenges were encountered and addressed during development. Correctly configuring the S3 event notification so that it triggered only on the `uploads/` prefix, rather than on the entire bucket, was necessary to avoid the recursive-invocation issue discussed in Section 2.8. Ensuring consistent color-mode handling within Pillow — particularly for images using palette or alpha-channel modes — required explicit conversion logic prior to saving output as JPEG. Coordinating the asynchronous nature of Lambda-based processing with a synchronous-feeling user interface

also required the introduction of the status-polling mechanism described in Section 3.5.

The current prototype is best characterized as a functional demonstration of the core event-driven architecture rather than a production-hardened system; its limitations are discussed in detail in the following chapter.



# Chapter 5

## Limitations and Future Scope

### 5.1 Current Limitations

- The system is currently deployed and evaluated at a prototype level, rather than as a hardened production service.
- Only a single thumbnail size is generated per uploaded image; no configurable or multiple-size output is currently supported.
- Batch or bulk image processing is not currently supported; each image is processed individually as it is uploaded.
- No user authentication or account system is currently implemented; the prototype does not distinguish between different users.
- No dedicated monitoring or logging dashboard has been built; visibility into Lambda execution currently relies on default AWS logging facilities.
- The system's processing tier is dependent on the continued availability of AWS services (S3, Lambda, IAM); no offline or alternative processing path exists.
- The frontend/backend relies on polling to detect thumbnail availability, rather than a push-based notification mechanism, which introduces some inherent latency and additional request overhead.

## 5.2 Future Enhancements

The following enhancements are identified as potential future scope and are *not* part of the current prototype:

1. Generation of multiple thumbnail sizes per uploaded image (e.g., small, medium, and large variants).
2. Support for batch or bulk image-processing workflows.
3. Output in modern image formats such as WebP or AVIF, in addition to or instead of JPEG.
4. Configurable compression quality settings for generated thumbnails.
5. Integration with a content delivery network (CDN) for faster global image delivery.
6. Introduction of a user authentication and account management system.
7. Support for user-specific image storage and access boundaries.
8. Maintaining a processing history log for uploaded images.
9. Development of a monitoring and logging dashboard for operational visibility.
10. Use of automatic S3 lifecycle policies to manage the retention of original images and thumbnails over time.
11. Fully serverless deployment of the frontend and backend components, in addition to the already-serverless processing tier.

# Chapter 6

## Conclusion

This report has presented the design, implementation, and evaluation of the Image Thumbnail Generator, a cloud-based, serverless image-processing prototype developed by team TechFusion. The project addressed the problem of automating thumbnail generation for uploaded images without imposing additional processing load on the main web application server, by adopting an event-driven architecture built around Amazon S3 and AWS Lambda.

The resulting system combines a lightweight HTML, CSS, and JavaScript frontend; a PHP 8 backend responsible for validation, storage orchestration, and presigned URL generation; and a Python-based AWS Lambda function that uses the Pillow library to perform the actual image resizing. Amazon S3 serves as the durable storage layer connecting these components, with S3 event notifications providing the trigger mechanism that automatically invokes image processing whenever a new image is uploaded, and AWS IAM providing least-privilege access control throughout.

Security was treated as an integral part of the design, incorporating environment-based credential management, private S3 storage, presigned URL access, layered client- and server-side validation, and randomized file-name generation, among other measures described in Section 2.7.

Through the development of this prototype, the team gained practical experience in designing decoupled, event-driven cloud architectures; integrating a traditional PHP web application with managed cloud services; and reasoning carefully about the security implications of a system that spans a browser, an application server, and serverless cloud infrastructure. The final prototype successfully demonstrates a complete, functional, automated pipeline for cloud-based image thumbnail generation, and provides a solid architec-

tural foundation upon which the future enhancements outlined in Chapter 5 could be built.

# Bibliography

- [1] Amazon Web Services, *Amazon Simple Storage Service (S3) Documentation*. Available: <https://docs.aws.amazon.com/s3/>.
- [2] Amazon Web Services, *AWS Lambda Documentation*. Available: <https://docs.aws.amazon.com/lambda/>.
- [3] Amazon Web Services, *AWS Identity and Access Management (IAM) Documentation*. Available: <https://docs.aws.amazon.com/iam/>.
- [4] Amazon Web Services, *AWS SDK for PHP Documentation*. Available: <https://docs.aws.amazon.com/sdk-for-php/>.
- [5] Python Software Foundation, *Python 3 Documentation*. Available: <https://docs.python.org/3/>.
- [6] Pillow Contributors, *Pillow (PIL Fork) Documentation*. Available: <https://pillow.readthedocs.io/>.
- [7] The PHP Group, *PHP Manual*. Available: <https://www.php.net/manual/en/>.